

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	V.Rakshana	Reg. No.:	192425137
Date:	27.08.2026	Duration:	60 minutes

Code of Conduct

I V.Rakshana(192425137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.Rakshana

Problem Statement 24: Smart Museum Artifact Preservation Platform

1. Introduction

The Smart Museum Artifact Preservation Platform is a software system designed to digitally manage, document, monitor, and preserve information related to museum artifacts. The platform can maintain artifact metadata, images and digital records, preservation status, environmental observations, conservation activities, and user access. Because the platform handles valuable cultural information, its software delivery process must be reliable, traceable, secure, and repeatable.

A CI/CD pipeline automates the movement of software from source-code commit through compilation/build, automated testing, code-quality and security checks, packaging, deployment to development and staging environments, and controlled release to production. The proposed pipeline uses GitHub for source control, GitHub Actions for automation, Docker for consistent packaging, SonarQube

for quality analysis, Trivy for container vulnerability scanning, and a controlled deployment model for development, staging, and production.

2. Project Requirements and CI/CD Needs

The platform requires continuous updates to the user interface, backend services, database schema, artifact-management features, authentication, search, reporting, and preservation workflows. These changes should be integrated without introducing defects into existing functions.

Requirement	CI/CD Need	Reason
Frequent feature updates	Automated build and test on every push/pull request	Detect integration problems early.
Artifact metadata integrity	Unit, integration, and database tests	Prevent incorrect or lost records.
Secure user access	SAST, dependency and secret scanning	Protect authentication and sensitive data.
Digital artifact files	Upload/API and storage integration tests	Ensure files and metadata remain usable.
Reliable releases	Docker image packaging and immutable versioning	Provide reproducible deployments.
Multiple environments	Dev → Staging → Production promotion	Separate experimentation from live operation.
Fast recovery	Rollback to previous known-good image/database version	Minimize service disruption.
Auditability	Git history, workflow logs, versioned artifacts	Track who changed and released software.

3. Proposed CI/CD Architecture

The pipeline follows a gated workflow. Developers commit changes to GitHub and create pull requests. Automated checks run before code is merged. After successful validation, the application is built into a Docker image, scanned, and stored in a container registry. Deployment first targets development and then staging. Production deployment requires an explicit approval gate and uses the same tested artifact.

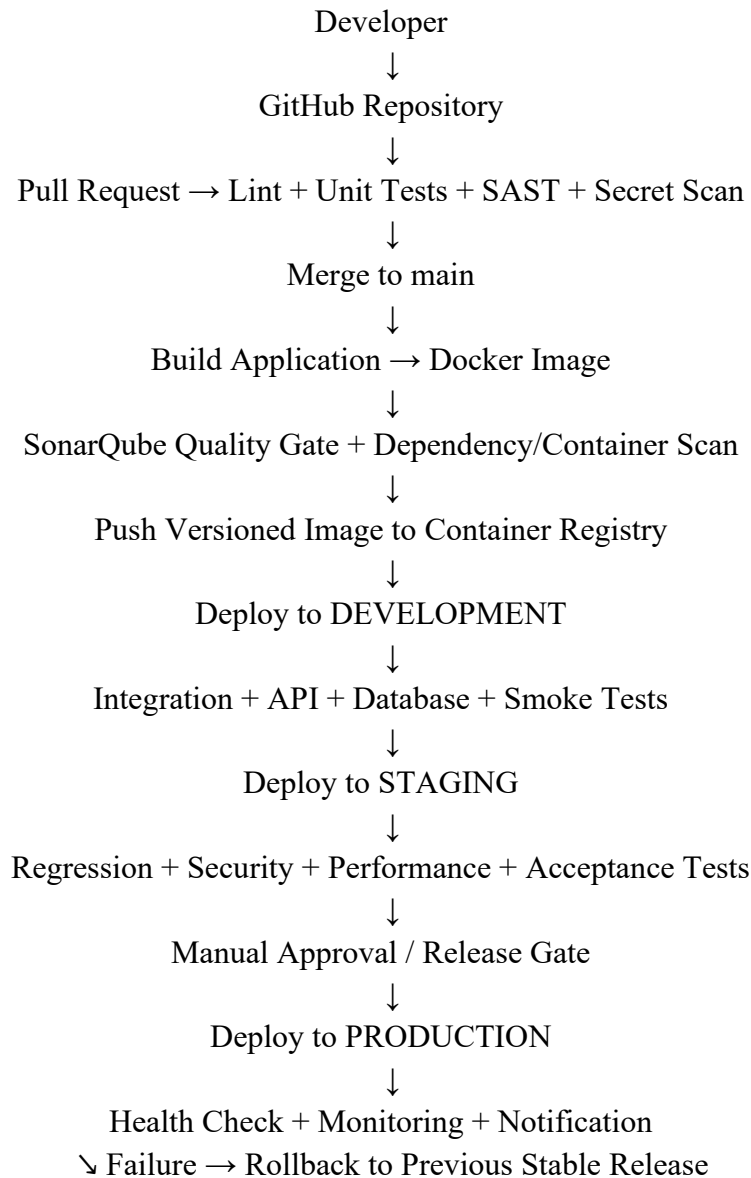


Figure 1: Proposed CI/CD pipeline workflow for the Smart Museum Artifact Preservation Platform.

4. Pipeline Stages, Tools and Justification

Stage	Tool / Technology	Main Activity	Justification
Source Code Management	Git + GitHub	Version control, branches, pull requests, review	Provides collaboration, traceability, and controlled merging.
CI Automation	GitHub Actions	Trigger workflows on push and pull request	Integrates naturally with GitHub and supports automated jobs.

Dependency Management	npm / package manager	Install locked dependencies	Improves reproducibility and dependency consistency.
Build	Node.js build process + Docker	Compile/build application and create image	Creates a portable, reproducible deployment unit.
Code Quality	SonarQube	Static analysis, bugs, code smells, coverage gate	Enforces measurable quality before release.
Security	Trivy + secret scanning	Scan dependencies, container image and repository	Detects common vulnerabilities and accidental secrets.
Testing	Jest / Supertest / Playwright	Unit, API/integration and end-to-end testing	Covers backend logic, APIs and user workflows.
Artifact Registry	GitHub Container Registry	Store versioned Docker images	Keeps deployable artifacts available for promotion.
Deployment	Docker Compose / container platform	Deploy to dev, staging and production	Provides consistent runtime environments.
Monitoring	Application logs + health checks	Verify availability after release	Detects deployment failures quickly.
Notifications	GitHub Actions notifications / email or chat	Report pipeline results	Keeps developers informed of failures and releases.

5. Source Code Management Strategy

The project repository should contain separate application folders, configuration files, database scripts, Docker configuration, tests, and CI workflow definitions. A protected main branch should be used for stable code. Developers can work on feature branches such as feature/artifact-search or feature/preservation-report.

A pull request should require successful automated checks before merging. Code review and branch protection reduce the chance of untested changes reaching the main branch.

6. Build and Packaging Process

After a successful merge, the pipeline installs dependencies using the project's lock file, runs the build command, and creates a Docker image. The image should be tagged using a unique version such as the Git commit SHA and, for releases, a semantic version such as v1.2.0. The exact image that passes staging should be promoted to production rather than rebuilt.

This approach improves reproducibility: the application, runtime dependencies, and startup configuration are captured in a consistent container image.

7. Automated Testing Strategy

Test Level	Example Tests	Pipeline Stage	Expected Result
Linting	ESLint / formatting checks	Pull request	No blocking style or syntax issues.
Unit Testing	Artifact service, authentication, validation, risk/status logic	Pull request / CI	All critical units pass.
API / Integration	CRUD operations, search, authentication, database connectivity	Development	Services work together correctly.
Database Testing	Schema migration and data constraints	Development / staging	Migrations execute safely and constraints hold.
End-to-End	Login → artifact search → artifact details → preservation update	Staging	Critical user journeys pass.
Security Testing	Dependency, secret and container vulnerability scans	CI / staging	No unacceptable vulnerabilities or exposed secrets.
Performance	Concurrent API requests and artifact search response time	Staging	System meets defined performance targets.
Smoke Testing	Health endpoint, login, core artifact retrieval	After every deployment	Deployment is operational.

Testing should follow a test pyramid: many fast unit tests, a smaller set of integration tests, and a focused set of end-to-end tests. Critical preservation and data-integrity functions should receive higher test coverage than non-critical UI functions.

8. Code Quality and Security Gates

The pipeline should fail when mandatory quality or security criteria are not met. SonarQube can enforce a quality gate covering bugs, vulnerabilities, maintainability issues, duplicated code, and test coverage. Dependency scanning checks third-party libraries for known vulnerabilities. Trivy can scan the final Docker image.

Secrets such as database passwords, API keys, cloud credentials, and signing keys must never be stored in the Git repository. They should be maintained as protected CI/CD secrets or environment-specific secret stores. Production deployment permissions should be restricted to authorized maintainers.

9. Environment and Deployment Strategy

Environment	Purpose	Deployment Policy	Data
Development	Developer integration and early verification	Automatic after successful main-branch CI	Synthetic/test data
Staging / Testing	Production-like validation and acceptance testing	Automatic promotion after development checks	Anonymized/synthetic data
Production	Live museum preservation platform	Manual approval after staging gates	Controlled production data with backups

Environment-specific configuration must be externalized. The same Docker image should be promoted from development to staging and then production. Production deployment should be performed only after all required checks pass.

10. Deployment Process

1. Developer creates a feature branch and commits code.
2. A pull request triggers linting, unit tests, security checks and code-quality analysis.
3. Reviewers inspect the change and merge it into the protected main branch only after required checks pass.
4. The CI workflow builds the application and Docker image.
5. The image is scanned for vulnerabilities and pushed to the container registry with an immutable version tag.

11. Rollback and Recovery Mechanism

Every production release should have a uniquely identifiable version. The platform should retain the previous known-good image so that a failed release can be replaced quickly. A rollback can redeploy the previous image and restart the service. Database migrations require additional care: migrations should be

backward-compatible where possible, and database backups should be taken before risky production changes.

For higher availability, the system can use a blue-green or rolling deployment model. A health check should verify application startup, database connectivity, authentication service availability, and a basic artifact retrieval operation before traffic is considered healthy.

12. Notifications and Monitoring

Pipeline status should be communicated to the development team through GitHub notifications and, where configured, email or a team chat channel. Notifications should identify the commit, branch, failed stage, test summary, and deployment status.

Post-deployment monitoring should include application logs, container status, health endpoints, error rates, response times, storage failures, and database connectivity. Alerts should be generated for critical failures.

13. CI/CD Pipeline Benefits for the Smart Museum Platform

- Reduces manual deployment effort and human error.
- Detects software defects early through automated testing.
- Improves software quality through mandatory quality gates.
- Protects the platform using automated security and vulnerability scanning.
- Creates reproducible deployments using Docker images.

14. Conclusion

The proposed CI/CD pipeline provides a structured and automated delivery process for the Smart Museum Artifact Preservation Platform. GitHub and GitHub Actions provide source control and workflow automation, Docker provides reproducible packaging, SonarQube enforces code quality, and security scanners identify vulnerabilities before release. Automated unit, integration, end-to-end, smoke, security, and performance testing provide layered validation.

The development–staging–production deployment model, combined with approval gates, monitoring, notifications, backups, and rollback, ensures that new features can be delivered quickly while protecting the reliability and integrity of museum artifact information.